ShinySwarm Benchmark Report

REST API Architecture (5-run aggregate)

Iva Gunjaca — University of Amsterdam

June 2026

1 Test Environment

Parameter	Value
Server	Hetzner CX32 — 8 vCPU, 16 GB RAM
Kubernetes	K3s single-node
R backends	Plumber APIs (HTTP)
State store	Redis (live) + PostgreSQL (checkpoints)
Relay	Spring Boot API gateway
Frontend	Angular + nginx reverse proxy
Load generator	k6 (remote)
Apps under test	8 collaborative R Shiny applications
Runs aggregated	5 (run-1 . . . run-5; run-03 trial excluded)

Validity Notes (read first)

- **Throughput (test 04) is excluded** as invalid: the 5 runs were not independent (server not reset between runs), so its numbers are not reported here.
- **Latency tests lack a per-VU breakdown.** Tests 01, 02, 03 and 08 ramp virtual users but tag metrics only with `app_name`, not `vu_bucket`. Their avg/median/p95 therefore aggregate all load levels: *the median reflects low-VU samples while the p95 reflects the high-VU tail.* Read median as the typical case, p95 as the loaded tail. This matters more for REST than for Kafka because the REST tail is dominated by the synchronous relay under load (see §**??**).
- **Per-app thresholds are baseline-calibrated** (10× each app's single-user baseline). A PASS means the app stayed within its own envelope under load, not that it was fast in absolute terms.

2 Summary

#	Benchmark	Result	Key Finding (5-run mean)
01	State Relay Latency	WARN	medians 21–90 ms, but heavy p95 tails (2–8 s); Advanced fails SLO; MLTrainer median 31 s (intrinsic)
02	Collaboration Latency	WARN	medians 60–110 ms; p95 2.6–7.6 s; Advanced fails SLO
03	Save & Restore Latency	WARN	median 10 ms; p95 up to 1.1 s; Geospatial, MonteCarlo, Advanced fail SLO
04	Throughput	**EXCLUDED — invalid (see validity notes)**	
05	Data Loss	PASS	0% loss, all runs, all apps
06	Cross-Contamination	PASS	0 contamination events, all runs
07	Session Lifecycle	WARN	0 errors; op median 1.0 s, p95 9.9 s; Advanced fails SLO
08	Multi-User Collaboration	PASS	all apps 20–57 ms; all within threshold
09	WebSocket Presence	PASS	connect ≈ 4 ms median, stable across VUs

Headline. At low load REST is fast for every app (relay medians of tens of milliseconds). Under concurrency the p95 tails grow large because the synchronous Spring Boot relay serialises requests: the cost shows up in the POST leg, not the poll (see §**??**). MLTrainer is a separate case — its ≈ 31 s median is intrinsic R compute, not relay overhead.

3 Detailed Results

3.1 01 — State Relay Latency

Round-trip: POST state → Spring Boot → Plumber computes → GET from Redis. Per-app `relay_total_ms`, mean of 5 runs.

App	avg	median	p95	threshold
Calculator	1.2 s	41 ms	7.7 s	PASS
Geospatial	807 ms	21 ms	4.9 s	PASS
DataExchange	491 ms	21 ms	2.0 s	PASS
MonteCarlo	503 ms	88 ms	2.2 s	PASS
Advanced	508 ms	30 ms	2.0 s	FAIL
Analytics	1.4 s	46 ms	8.0 s	PASS
ClimateAnomaly	1.4 s	44 ms	8.8 s	PASS
MLTrainer	38.7 s	30.8 s	78.0 s	PASS*

*MLTrainer's threshold (143 s) is calibrated to its ≈ 14 s single-user compute, so it passes despite a 31 s median. Advanced fails its 730 ms SLO due to its loaded p95.

Note the median/p95 gap. Most apps have a median of tens of ms but a p95 of several seconds. That gap is the VU ramp: low-VU samples are fast, high-VU samples are slow, and they are pooled (no `vu_bucket` tag). MLTrainer is the exception — its median itself is large because the app's computation is large.

3.2 Where the time goes: POST vs. POLL

REST decomposes the relay into the POST (synchronous call through Spring Boot to Plumber) and the POLL (read result from Redis). Global values (run-2, representative):

Stage	median	p95
`relay_post_ms` (POST via Spring Boot)	29 ms	30.0 s
`relay_poll_ms` (read from Redis)	5 ms	1.5 s

Unlike Kafka (where the wait-for-result poll dominates), in REST the **POST leg** carries the tail: it is synchronous and runs through the central Spring Boot relay, which serialises under load. The Redis poll stays cheap. This is the architectural signature of the REST design — a single relay point that becomes the bottleneck as concurrency rises.

3.3 02 — Collaboration Latency

Host posts state; guest polls until the update appears. Per-app `collab_propagation_ms`, mean of 5 runs.

App	avg	median	p95	threshold
Calculator	640 ms	71 ms	2.6 s	**PASS**
Geospatial	915 ms	65 ms	4.5 s	**PASS**
DataExchange	757 ms	71 ms	2.8 s	**PASS**
MonteCarlo	818 ms	85 ms	4.7 s	**PASS**
Advanced	980 ms	107 ms	4.3 s	**FAIL**
Analytics	827 ms	97 ms	5.0 s	**PASS**
ClimateAnomaly	617 ms	60 ms	3.1 s	**PASS**
MLTrainer	1.6 s	343 ms	7.6 s	**PASS**[*]

Median propagation is well under 350 ms for every app; the multi-second p95 is again the loaded tail.

3.4 03 — Save & Restore Latency

PostgreSQL snapshot cycle. Per-app `save_to_db_ms`, mean of 5 runs.

App	avg	median	p95	threshold
Calculator	251 ms	10 ms	590 ms	**PASS**
Geospatial	365 ms	10 ms	717 ms	**FAIL**
DataExchange	251 ms	10 ms	1.1 s	**PASS**
MonteCarlo	304 ms	10 ms	844 ms	**FAIL**
Advanced	296 ms	10 ms	269 ms	**FAIL**
Analytics	271 ms	10 ms	407 ms	**PASS**
ClimateAnomaly	132 ms	10 ms	47 ms	**PASS**
MLTrainer	265 ms	10 ms	201 ms	**PASS**

Save is 10 ms at the median for every app, but three apps cross their (tight) save SLOs on the loaded tail. The PostgreSQL connection pool is the shared contention point under concurrency.

3.5 05 — Data Loss

Metric	Value (all 5 runs)
Loss rate	0%
Checks passed	105 / 105 (100%) per run

Threshold `dataloss_pct` $<$ `5%`: **PASS**.

3.6 06 — Cross-Contamination

Metric	Value (all 5 runs)
Contamination events	0

Threshold `contamination_events == 0`: **PASS**.

3.7 07 — Session Lifecycle

Metric	Value (run-1, representative)
Lifecycle errors	0 / 1,825 (0%)
`lifecycle_op_ms` median	1.02 s
`lifecycle_op_ms` p95	9.85 s

No errors, but Advanced crosses its SLO on the loaded tail.

3.8 08 — Multi-User Collaboration

One host posts; multiple guests poll. Per-app `multi_propagation_all_ms`, mean of 5 runs.

App	avg	median	p95	threshold
Calculator	45 ms	38 ms	38 ms	**PASS**
Geospatial	28 ms	22 ms	22 ms	**PASS**
DataExchange	40 ms	33 ms	33 ms	**PASS**
MonteCarlo	37 ms	31 ms	31 ms	**PASS**
Advanced	57 ms	57 ms	57 ms	**PASS**
Analytics	38 ms	20 ms	20 ms	**PASS**
ClimateAnomaly	29 ms	26 ms	26 ms	**PASS**
MLTrainer	23 ms	20 ms	20 ms	**PASS**

All apps pass comfortably. (Note: this test uses a small fixed VU count per group, so it does not stress the relay the way the ramped tests do — which is why the heavy apps look fast here.)

3.9 09 — WebSocket Presence

Metric	median	p95	vs. VU load
`ws_connect_ms`	4 ms	6 ms	flat (no degradation)
`ws_join_roundtrip_ms`	4 ms	6 ms	flat

Connect stays ≈ 4 ms from 1 to 50 VUs.

4 Analysis

REST is correct on every reliability dimension: zero data loss, zero cross-contamination, zero lifecycle errors across all five runs.

On latency, REST is fast at the median for every app (tens of ms on relay and propagation). The characteristic weakness is the **loaded tail**: p95 climbs to several seconds because the synchronous Spring Boot relay serialises requests under concurrency. The POST/POLL split (§**??**) localises this to the POST leg — the relay itself — while the Redis read stays cheap. This is the expected trade-off of a single central relay point.

MLTrainer is a distinct case: its large median (31 s on relay) is intrinsic R computation, the same application property seen under Kafka, not a REST relay cost.

Comparison hook (REST vs. Kafka). Both architectures are equally correct. The latency *shape* differs: under Kafka the heavy apps pay in the POLL (waiting for their own compute) while light apps are flat at ≈ 265 ms; under REST the cost appears in the POST leg as a loaded tail that affects all apps as concurrency rises. A like-for-like latency-vs-VU comparison requires the VU tagging fix on tests 01/02/03/08.

Caveats. (1) Throughput excluded as invalid. (2) Latency tests aggregate across VU levels (no `vu_bucket` tag); medians and p95 should be read as typical-case and loaded-tail respectively, not as one operating point.